

Implementation Plan & Engineering Execution Roadmap

App Name: CloudGuard GRC (AntiGravity Platform Architecture)[cite: 1]

Document Version: 1.0.0 (Sequential Development Blueprint & AI Agent Directives)

Target Stack: React.js (TypeScript), FastAPI (Python), AWS RDS PostgreSQL, AWS SQS/Lambda[cite: 1]

1. Development Execution Strategy & Build Sequence

To prevent architectural drift and ensure stable dependency management, an AI coding agent or development team **MUST** follow this build sequence in strict numerical order. Each phase builds upon verified artifacts from previous steps.

Seq #	Phase Name	Primary Tech Stack Involved	Core Focus & Goal
01	Project Initialization & Setup	Git, Docker, Node.js, Python 3.11+	Repository structure, linters, Docker Compose setup.
02	Database Migration & RLS Engine	PostgreSQL, SQLAlchemy, Alembic	Tables schema creation, Row-Level Security policies.
03	Auth Engine & Tenant Middleware	FastAPI, PyJWT, Passlib (Argon2)	JWT auth, user login/signup, tenant context middleware.
04	Core Frontend Shell & Design System	React, Vite, TailwindCSS, shadcn/ui	Theme configuration, App Layout shell, navigation state.
05	AWS Onboarding & Scanner Pipeline	boto3 , AWS STS, SQS, Step Functions	Role assumption (STS), asset discovery, async scan worker.
06	Security Rules Engine & GRC Mapper	Python Rules Engine, Rego/OPA	CIS benchmarks, secret detection, DPDPA/GDPR mappings.
07	UI Dashboard & Remediation Drawer	TanStack Query, Recharts, WeasyPrint	Risk gauges, findings grid, Terraform fix code, PDF reports.
08	Integrations & Webhook Engine	Slack REST API, Jira SDK, Amazon SNS	Real-time alerts, ITSM ticket creation.
09	Automated Testing & Security Audit	Pytest, LocalStack, Playwright	Unit tests, IAM sandbox mocks, E2E test scripts.
10	Cloud Deployment & Final Polish	Terraform, AWS ECS Fargate, CloudFront	Production deployment, zero-downtime CI/CD.

2. Phase-by-Phase Technical Implementation Steps

PHASE 01 Environment Setup & Project Initialization

1. Initialize a monorepo or dual-directory structure: `/apps/web` (Frontend) and `/apps/api` (Backend).
2. Set up `/apps/api` using Python 3.11+ with `poetry` or `pipenv`. Install `fastapi`, `uvicorn`, `boto3`, `pydantic-settings`, and `sqlalchemy`.
3. Set up `/apps/web` using Vite + React with TypeScript. Configure `TailwindCSS` and install `lucide-react`, `axios`, and `zustand`.
4. Create a local `docker-compose.yml` file containing PostgreSQL v15, Redis, and LocalStack (for offline AWS API emulation).
5. Configure ESLint, Prettier, and `black` / `ruff` for code formatting enforcement across commits.

Phase 01 Deliverable: Runnable Docker Compose environment with responding frontend (Vite) and backend (FastAPI `/health` endpoint).

PHASE 02 Database Layer, Models & Isolation Rules

1. Set up Alembic inside `/apps/api` for managing SQL schema migrations.
2. Implement SQLAlchemy ORM models matching the database schema document:
 - `Tenant`, `User`, `Session`, `CloudAccount`, `Scan`, `Finding`, `AuditLog`.
3. Create database migration scripts to apply primary keys, foreign key constraints, and performance indexes.
4. Write SQL migration scripts to enable PostgreSQL Row-Level Security (RLS) policies on the `findings` and `cloud_accounts` tables.

Phase 02 Deliverable: Executable DB migration scripts creating all 7 core tables with index definitions and verified RLS isolation rules.

PHASE 03 Authentication, Session & Role Authorization

1. Implement secure user password hashing routines using `Argon2id`.
2. Build Auth REST Endpoints: `POST /api/v1/auth/register` and `POST /api/v1/auth/login`.
3. Implement JWT token generation (Access Token: 15-min expiry, Refresh Token: 7-day expiry with database session tracking in `sessions` table).
4. Create FastAPI Request Middleware (`TenantContextMiddleware`) that extracts JWT bearer tokens, validates tenant ownership, and executes `SET LOCAL app.current_tenant_id = ...` on database transactions.
5. Create Role-Based Access Control (RBAC) decorators: `@require_role(["SUPER_ADMIN", "SECURITY_ANALYST"])`.

Phase 03 Deliverable: Functional authentication pipeline issuing secure JWTs with enforced multi-tenant database session context.

PHASE 04 Frontend Shell, Navigation & UI System

1. Initialize `shadcn/ui` component primitives (Button, Card, Dialog, Table, Badge, Tabs, DropdownMenu).
2. Configure Tailwind CSS design tokens matching the UI/UX Brief:
 - Canvas: `#090d16`, Surface: `#131c2e`, Brand: `#2563eb`, Critical: `#ef4444`.
3. Build standard SaaS Shell layout: Collapsible left sidebar navigation, top header bar with tenant selector, user profile menu, and alert bell.
4. Configure React Router v6 route guards for authenticated vs. public routes.

Phase 04 Deliverable: Responsive web dashboard shell with dark-mode aesthetic, route guarding, and component tokens ready for data binding.

PHASE 05 AWS Onboarding & Asynchronous Scan Worker Engine

1. Create AWS Onboarding endpoint: `POST /api/v1/accounts/onboard` . Generate unique `ExternalID` for cross-account protection[cite: 1].
2. Build validation worker using `boto3` : Call `sts:AssumeRole` against target IAM Role ARN to verify read-only connection status[cite: 1].
3. Construct AWS Scan Engine Task: Worker assumes client role and fetches configurations via SDK calls:
 - `s3:GetBucketAcl` , `iam:ListAccessKeys` , `ec2:DescribeSecurityGroups` , `kms:ListKeys` , `cloudtrail:DescribeTrails` [cite: 1].
4. Set up background task runner (Celery + Redis or AWS SQS + Lambda) to execute scans asynchronously without blocking REST API handlers.

Phase 05 Deliverable: Asynchronous scan worker capable of assuming Read-Only IAM roles and ingesting AWS JSON configurations into database storage.

PHASE 06 Core Security Rules, Secret Detection & GRC Mappings

1. Build Rule Evaluation Engine in Python. Write initial suite of CIS AWS Foundations v1.2 checks:
 - `RULE-AWS-001` : Root account MFA check.
 - `RULE-AWS-002` : S3 Bucket Public Access Block check.
 - `RULE-AWS-003` : Security Group open SSH (Port 22 to `0.0.0.0/0`) check.
 - `RULE-AWS-004` : Unused/Stale IAM Access Keys (>90 days) check.
2. Implement Regex-based Secret Scanner analyzing S3 metadata and environment configurations for plaintext AWS keys or tokens[cite: 1].
3. Implement GRC Framework Mapper: Annotate findings with compliance control IDs (e.g., CIS AWS 2.1.1 → DPDPA Section 8 / GDPR Art. 32)[cite: 1].
4. Build Remediation Generator: Attach Terraform HCL code blocks and AWS CLI fix strings to generated finding payloads[cite: 1].

Phase 06 Deliverable: Fully operational rule evaluation engine producing structured finding records complete with GRC tags and Terraform fix code.

PHASE 07 UI Data Binding, Analytics Dashboard & Audit Reports

1. Bind Dashboard UI (`SCR-03`) to backend endpoints using `TanStack Query` [cite: 1]:
 - Render `ThreatScore` gauge (0–100 scale algorithm based on open findings).
 - Render `Recharts` risk distribution donut chart and compliance progress bars[cite: 1].
2. Build Interactive Findings Table (`SCR-06`) with sorting by severity, search filtering, and pagination[cite: 1].
3. Build Finding Detail Drawer (`SCR-07`): Slide-over drawer with copyable Terraform remediation code blocks[cite: 1].
4. Implement PDF Export Engine: Build backend endpoint `GET /api/v1/reports/export` using `WeasyPrint` to compile executive compliance PDF reports[cite: 1].

Phase 07 Deliverable: Fully functional visual dashboard displaying live AWS scan results, interactive fix drawers, and downloadable PDF compliance reports.

PHASE 08 Alerting Engine & Third-Party Integrations

1. Implement Notification Dispatcher: Send real-time Slack incoming webhook messages when `CRITICAL` findings are detected[cite: 1].
2. Implement Email Notification Service using AWS SES / SendGrid for scheduled daily scan summaries.
3. Build Jira REST API Integration: Allow users to click "Create Jira Ticket" directly from the Finding Detail Drawer[cite: 1].

Phase 08 Deliverable: Functional alert system firing real-time notifications to Slack webhooks and creating Jira issue tickets for cloud risks.

PHASE 09 Automated Testing, Security Auditing & QA

1. Write Backend Unit Tests (`pytest`): Test Python rules engine against static mock JSON AWS configuration payloads.
2. Write Integration Tests: Execute end-to-end scan flows against a sandbox AWS account or `LocalStack` instance containing intentional misconfigurations.
3. Execute Security Audit: Test multi-tenant isolation by attempting cross-tenant API requests (verifying RLS blocks unauthorized access).
4. Write Frontend E2E Tests (Playwright): Test user login, account onboarding wizard, and report generation triggers.

Phase 09 Deliverable: Automated test suite achieving >80% code coverage, confirming zero multi-tenant data leakage.

PHASE 10 Production Infrastructure & Deployment

1. Write Terraform IaC scripts defining production infrastructure on AWS:
 - VPC with public/private subnets, AWS RDS PostgreSQL instance, AWS SQS queues, AWS ECS Fargate cluster[cite: 1].
2. Build Production Docker Images for Web Frontend (Nginx static bundle) and Backend API.
3. Configure AWS CloudFront CDN for distribution of React frontend with HTTPS/TLS certificates.
4. Set up GitHub Actions CI/CD Pipeline: Automatically run tests, build Docker containers, push to AWS ECR, and deploy to ECS Fargate upon git push to `main` branch[cite: 1].
5. Configure AWS CloudWatch alarms and Sentry crash reporting for runtime error monitoring[cite: 1].

Phase 10 Deliverable: Deployed, production-ready CloudGuard GRC SaaS application running on AWS infrastructure with continuous deployment pipelines.